

Banco de Dados II — Aula 07

Transações, propriedades ACID e confirmação em duas fases (2PC)

Prof. Me. Alysson Martins Bruno Bacharelado em Sistemas de Informação — UNITINS 26 de setembro de 2026

Sumário

1. [Introdução](#)
 2. [Conceito de transação](#)
 3. [Problemas que as transações resolvem](#)
 4. [Contexto histórico](#)
 5. [Propriedades ACID](#)
 6. [Controle de transação no PostgreSQL](#)
 7. [Consistência em ambientes distribuídos e o protocolo 2PC](#)
 8. [Laboratório no iximiuz Labs](#)
 9. [Pontos-chave](#)
 10. [Referências](#)
-

1. Introdução

Nas Aulas 05 e 06, o banco de dados aprendeu a executar regras por conta própria: funções, procedimentos e gatilhos passaram a morar **dentro** do servidor. Falta responder a uma pergunta anterior a todas essas: o que acontece quando uma operação que envolve vários passos é interrompida no meio?

Um sistema de informação confiável não depende apenas de consultas rápidas ou de um modelo bem normalizado. Em ambientes reais, vários usuários alteram os mesmos dados ao mesmo tempo, falhas de hardware e de software acontecem sem aviso e operações importantes envolvem mais de uma tabela — ou mais de um servidor. É nesse contexto que o conceito de **transação** se torna central (SILBERSCHATZ; KORTH; SUDARSHAN, 2020).

Este texto apresenta:

- o conceito de transação e os problemas que ela resolve;
- o contexto histórico em que ele foi formalizado;
- as quatro propriedades **ACID** e os mecanismos do PostgreSQL que garantem cada uma;
- os comandos de controle BEGIN, COMMIT, ROLLBACK e SAVEPOINT;
- a consistência em ambientes distribuídos e o **protocolo de confirmação em duas fases (2PC)**;

- um laboratório em sete atividades independentes no iximiuz Labs.

Os temas de controle de concorrência — MVCC, bloqueios, anomalias e níveis de isolamento — são o assunto da Aula 08. Aqui o isolamento aparece apenas como uma das quatro propriedades.

1.1 O cenário usado em todo o texto

Todos os exemplos usam a **Cooperativa Cerrado**, uma cooperativa de crédito fictícia com agências em Palmas e em Gurupi. A tabela é a mesma do laboratório:

```
CREATE TABLE conta (  
    id          integer          PRIMARY KEY,  
    titular     varchar(40)      NOT NULL,  
    cidade      varchar(30)      NOT NULL,  
    saldo       numeric(10,2)    NOT NULL CHECK (saldo >= 0)  
);  
  
INSERT INTO conta (id, titular, cidade, saldo) VALUES  
    (1, 'Ana',    'Palmas', 1000.00),  
    (2, 'Bruno',  'Palmas',  500.00),  
    (3, 'Carla',  'Gurupi',  200.00),  
    (4, 'Davi',   'Gurupi',   50.00);
```

Dois detalhes importam:

- a restrição CHECK (saldo >= 0) impede que qualquer conta fique negativa;
- a soma dos saldos é **R\$ 1.750,00**. Uma transferência apenas muda o dinheiro de lugar; portanto, **nenhuma transferência pode alterar esse total**.

2. Conceito de transação

2.1 Definição

Uma **transação** é uma sequência de operações de banco de dados — leituras e escritas — tratada como uma **unidade lógica indivisível de trabalho** (SILBERSCHATZ; KORTH; SUDARSHAN, 2020). O princípio é simples: ou todas as operações são executadas e os seus efeitos ficam gravados, ou nenhuma delas deixa efeito.

Do ponto de vista do SGBD, uma transação tem:

- um **início**, explícito (BEGIN ou START TRANSACTION) ou implícito (um comando isolado, no modo *autocommit*);
- um **término**, que pode ser uma **confirmação** (COMMIT) ou um **desfazimento** (ROLLBACK).

2.2 Exemplo motivador: a transferência

Para transferir R\$ 300,00 da Ana para o Davi, são necessários dois comandos:

```
-- Passo 1: debitar da conta da Ana
UPDATE conta SET saldo = saldo - 300 WHERE id = 1;

-- Passo 2: creditar na conta do Davi
UPDATE conta SET saldo = saldo + 300 WHERE id = 4;
```

Isoladamente, cada comando é simples. O problema aparece quando o sistema falha depois do passo 1 e antes do passo 2: a Ana perdeu R\$ 300,00, o Davi não recebeu nada e o total da cooperativa caiu para R\$ 1.450,00. O banco ficou **inconsistente** — o dinheiro sumiu.

A solução é envolver os dois comandos em **uma única transação**:

```
BEGIN;
UPDATE conta SET saldo = saldo - 300 WHERE id = 1;
UPDATE conta SET saldo = saldo + 300 WHERE id = 4;
COMMIT;
```

Agora, se a falha ocorrer antes do COMMIT, o SGBD garante que **nenhum** dos dois comandos produza efeito.

2.3 Estados de uma transação

Segundo Silberschatz, Korth e Sudarshan (2020), uma transação passa pelos seguintes estados:

Estado	Descrição
Ativa (<i>active</i>)	estado inicial; as operações estão sendo executadas
Parcialmente confirmada (<i>partially committed</i>)	a última operação foi executada; falta gravar a confirmação
Confirmada (<i>committed</i>)	todas as operações foram concluídas e gravadas
Falha (<i>failed</i>)	um erro impediu a continuação normal
Abortada (<i>aborted</i>)	tudo foi desfeito; o banco voltou ao estado anterior ao início

Da **ativa**, a transação segue para **parcialmente confirmada** (quando a última operação termina) ou para **falha** (quando ocorre um erro). Da parcialmente confirmada, vai para **confirmada** quando a gravação termina; da falha, vai para **abortada** quando o desfazimento termina. Confirmada e abortada são estados **finais**: não há volta a partir deles.

No PostgreSQL, o estado de **falha** é visível: depois de um erro dentro de uma transação, o servidor responde a qualquer comando com `current transaction is aborted, commands ignored until end of transaction block` (Seção 6.4).

3. Problemas que as transações resolvem

As transações foram concebidas contra dois inimigos: as **falhas** e o **acesso concorrente**.

3.1 Falhas

Sistemas computacionais estão sujeitos a falhas de várias naturezas (SILBERSCHATZ; KORTH; SUDARSHAN, 2020):

- **Falhas de transação:** erros lógicos (violação de restrição, divisão por zero) ou cancelamento pelo usuário.
- **Falhas de sistema** (*crash*): queda de energia, travamento do sistema operacional, processo do SGBD derrubado.
- **Falhas de mídia:** perda ou corrupção do dispositivo de armazenamento.

Nas duas primeiras, o SGBD deve garantir que o banco volte a um estado consistente: o que foi confirmado permanece; o que não foi confirmado não deixa rastro. As falhas de mídia exigem cópias de segurança, tema da Aula 13.

3.2 Acesso concorrente

Em ambientes multiusuário, várias transações executam ao mesmo tempo. Sem controle, uma pode ler dados que a outra ainda não confirmou, ou duas podem alterar o mesmo dado e uma **apagar** a alteração da outra — a chamada **atualização perdida** (*lost update*). Imagine dois caixas creditando, ao mesmo tempo, R\$ 10,00 na conta do Bruno: ambos leem R\$ 500,00, ambos escrevem R\$ 510,00, e um dos créditos desaparece.

Esses problemas — e os mecanismos que o PostgreSQL usa para evitá-los — são o tema da Aula 08.

4. Contexto histórico

4.1 Jim Gray e o System R

A formalização do conceito de transação está ligada ao trabalho de **Jim Gray** na IBM, durante a década de 1970. O projeto **System R**, ancestral direto da linguagem SQL e dos SGBDs relacionais modernos, foi o ambiente em que Gray e seus colegas desenvolveram os mecanismos de controle de concorrência e de recuperação de falhas que fundamentam a teoria das transações até hoje (GRAY; REUTER, 1992).

Em 1978, nas notas de curso *Notes on Data Base Operating Systems*, Gray descreveu o **protocolo de confirmação em duas fases**, que coordena uma transação entre vários sistemas (GRAY, 1978) e que é apresentado na Seção 7. Em 1998, recebeu o **Prêmio Turing**, a mais alta distinção da Ciência da Computação, em grande parte pelo impacto de seu trabalho em processamento de transações.

4.2 Härder e Reuter

Em 1983, os pesquisadores alemães **Theo Härder** e **Andreas Reuter** publicaram o artigo *Principles of Transaction-Oriented Database Recovery*, na revista *ACM Computing Surveys*. Nele, **cunharam o acrônimo ACID** para nomear as quatro propriedades que um SGBD deve garantir para que suas transações sejam confiáveis (HÄRDER; REUTER, 1983).

O artigo sintetizou conceitos que estavam dispersos na literatura e criou uma linguagem comum para projetar, documentar e comparar SGBDs. Ainda hoje, é uma das referências mais citadas da área.

5. Propriedades ACID

As quatro propriedades formam o **contrato** entre o SGBD e quem escreve a aplicação:

Letra	Propriedade	Promessa
A	Atomicidade	tudo ou nada
C	Consistência	de um estado válido para outro estado válido
I	Isolamento	uma transação não interfere na outra
D	Durabilidade	o que foi confirmado permanece, mesmo após falha

5.1 Atomicidade (*atomicity*)

A **atomicidade** estabelece que a transação é indivisível: ou todas as suas operações produzem efeito, ou nenhuma produz (SILBERSCHATZ; KORTH; SUDARSHAN, 2020).

Considere uma transferência de R\$ 80,00 do Davi para a Carla em que o crédito é feito **antes** do débito:

```
BEGIN;  
UPDATE conta SET saldo = saldo + 80 WHERE id = 3;    -- crédito em Carla: UPDATE 1  
UPDATE conta SET saldo = saldo - 80 WHERE id = 4;    -- débito em Davi: ERRO  
COMMIT;
```

O segundo UPDATE falha, porque o Davi tem apenas R\$ 50,00 e o saldo ficaria negativo. Ao final, a Carla continua com R\$ 200,00: o crédito, que havia funcionado, **também foi desfeito**. O COMMIT, nesse caso, responde ROLLBACK (Seção 6.4).

A atomicidade só vale para as operações que estão **na mesma transação**. Sem BEGIN, cada comando é uma transação sozinha (Seção 6.1), e o SGBD não tem como saber que os dois UPDATE formam uma unidade.

5.2 Consistência (*consistency*)

A **consistência** determina que a transação leve o banco de um **estado consistente** a outro **estado consistente** (ELMASRI; NAVATHE, 2018). Um estado é consistente quando todas as regras de integridade estão satisfeitas.

A consistência é uma **responsabilidade compartilhada**:

- o **SGBD** garante as regras declaradas no esquema — NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK — e as regras programadas em gatilhos (Aula 06), recusando a operação que as viole;
- o **desenvolvedor** garante as regras do negócio que não estão declaradas. O SGBD recusa uma conta negativa, mas não impede um programa de debitar a Ana e esquecer de creditar o Davi. Manter o total da cooperativa é tarefa de quem escreve a transação.

No exemplo da Seção 5.1, quem barrou a operação foi a restrição CHECK (`saldo >= 0`) — a consistência — e quem garantiu que nada ficasse pela metade foi a atomicidade. As duas propriedades trabalham juntas.

5.3 Isolamento (*isolation*)

O **isolamento** estabelece que transações executadas ao mesmo tempo devem se comportar como se executassem **uma depois da outra** (GRAY; REUTER, 1992). Cada transação deve enxergar o banco como se fosse a única em execução.

No PostgreSQL, isso aparece de duas formas que o laboratório permite observar:

- uma sessão **não enxerga** o que outra sessão alterou e ainda não confirmou;
- quando duas transações tentam alterar **a mesma linha**, a segunda **espera** a primeira terminar.

O isolamento perfeito tem um custo alto de desempenho. Por isso o padrão SQL define **níveis de isolamento**, que trocam rigor por desempenho, e o PostgreSQL implementa o isolamento com o **MVCC** (*Multi-Version Concurrency Control*), que mantém várias versões de uma mesma linha. Níveis, anomalias e MVCC são o tema da Aula 08.

5.4 Durabilidade (*durability*)

A **durabilidade** garante que, depois do COMMIT, os efeitos da transação **permanecem**, mesmo que o servidor falhe no instante seguinte (SILBERSCHATZ; KORTH; SUDARSHAN, 2020).

No PostgreSQL, a durabilidade é garantida pelo **WAL** (*Write-Ahead Log*, registro de escrita antecipada), apresentado na Aula 04 como parte da organização física do servidor (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026e):

1. toda alteração é descrita primeiro no WAL e só depois aplicada aos arquivos de dados — daí o nome *escrita antecipada*;
2. quando a transação executa COMMIT, o PostgreSQL força a gravação em disco dos registros do WAL daquela transação antes de responder ao cliente;

3. se o servidor cai, ao religar ele percebe que não foi desligado corretamente e **refaz** (*redo*) o WAL a partir do último ponto de verificação (*checkpoint*), reconstruindo tudo o que havia sido confirmado.

E o que não foi confirmado? O PostgreSQL não precisa “desfazer” nada no disco: cada versão de linha carrega o número da transação que a criou, e as versões criadas por transações que não chegaram ao COMMIT simplesmente **não são visíveis** para ninguém. Mais tarde, o VACUUM recupera esse espaço.

A Atividade 4 do laboratório derruba o servidor com `podman kill` e mostra, no log, as linhas `database system was not properly shut down; automatic recovery in progress` e `redo starts at:` é a durabilidade em funcionamento.

5.5 Quem garante cada propriedade no PostgreSQL

Propriedade	Mecanismo	Como se observa
Atomicidade	a transação inteira é marcada como abortada; as versões que ela criou ficam invisíveis	o COMMIT de uma transação com erro responde ROLLBACK
Consistência	restrições de integridade e gatilhos	<code>violates check constraint</code>
Isolamento	MVCC e bloqueios de linha	a outra sessão não vê a alteração pendente; quem altera a mesma linha espera
Durabilidade	WAL gravado em disco no COMMIT; recuperação por <i>redo</i>	<code>automatic recovery in progress</code> no log depois de uma queda

6. Controle de transação no PostgreSQL

Os comandos que delimitam e controlam transações formam a **TCL** (*Transaction Control Language*):

Comando	Efeito
BEGIN ou START TRANSACTION	abre uma transação
COMMIT	confirma tudo o que foi feito desde o BEGIN
ROLLBACK	desfaz tudo o que foi feito desde o BEGIN
SAVEPOINT nome	marca um ponto seguro dentro da transação
ROLLBACK TO SAVEPOINT nome	desfaz o que foi feito depois do ponto seguro, sem encerrar a transação
RELEASE SAVEPOINT nome	descarta o ponto seguro

6.1 Autocommit: o comportamento padrão

Sem BEGIN, o PostgreSQL executa **cada comando como uma transação independente**, confirmada automaticamente ao terminar. Esse modo é chamado de *autocommit* e é o padrão do psql e do pgAdmin (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026a).

A consequência é que um comando executado sem BEGIN **não pode ser desfeito**:

```
UPDATE conta SET saldo = saldo - 10 WHERE id = 1;
ROLLBACK;
```

```
UPDATE 1
WARNING:  there is no transaction in progress
ROLLBACK
```

O aviso deixa claro que não havia transação para desfazer: os R\$ 10,00 já saíram da conta da Ana.

6.2 BEGIN e COMMIT

BEGIN (ou o sinônimo do padrão SQL, START TRANSACTION) abre uma transação; os comandos seguintes pertencem a ela até o COMMIT:

```
BEGIN;
UPDATE conta SET saldo = saldo - 100 WHERE id = 1;    -- débito em Ana
UPDATE conta SET saldo = saldo + 100 WHERE id = 2;    -- crédito em Bruno
COMMIT;

SELECT id, titular, saldo FROM conta ORDER BY id;
```

id	titular	saldo
1	Ana	900.00
2	Bruno	600.00
3	Carla	200.00
4	Davi	50.00

Somente depois do COMMIT as alterações se tornam visíveis para as demais sessões e passam a ser duráveis.

6.3 ROLLBACK

ROLLBACK encerra a transação desfazendo **tudo** o que foi feito desde o BEGIN:

```
BEGIN;
UPDATE conta SET saldo = saldo - 50 WHERE id = 2;
UPDATE conta SET saldo = saldo + 50 WHERE id = 3;
SELECT id, titular, saldo FROM conta ORDER BY id;
-- dentro da transação: Bruno 550.00 e Carla 250.00

ROLLBACK;
SELECT id, titular, saldo FROM conta ORDER BY id;
-- depois: Bruno 600.00 e Carla 200.00
```

Dentro da transação, a própria sessão enxerga as alterações; depois do ROLLBACK, elas desaparecem juntas. Nenhuma outra sessão chegou a vê-las.

6.4 Erro dentro da transação: o estado abortado

Quando um comando falha dentro de uma transação, o PostgreSQL coloca a transação no estado de **falha** e passa a recusar qualquer comando:

```
BEGIN;  
UPDATE conta SET saldo = saldo + 80 WHERE id = 3;  
UPDATE conta SET saldo = saldo - 80 WHERE id = 4;
```

```
UPDATE 1  
ERROR:  new row for relation "conta" violates check constraint  
"conta_saldo_check"  
DETAIL:  Failing row contains (4, Davi, Gurupi, -30.00).
```

```
SELECT id, titular, saldo FROM conta ORDER BY id;
```

```
ERROR:  current transaction is aborted, commands ignored until end of transaction  
block
```

A única saída é encerrar a transação. E mesmo que se peça COMMIT, o servidor responde ROLLBACK:

```
COMMIT;
```

```
ROLLBACK
```

Esse comportamento é a atomicidade em ação: uma transação que falhou não pode ser confirmada, nem em parte. Na programação de aplicações, a lição é **conferir sempre o resultado do COMMIT**: ele pode não ter confirmado nada.

6.5 SAVEPOINT: desfazer só um pedaço

Às vezes não se quer perder a transação inteira por causa de um erro. O SAVEPOINT marca um **ponto seguro** dentro da transação; o ROLLBACK TO SAVEPOINT volta até ele, desfazendo apenas o que veio depois, e a transação **continua** (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026b).

No exemplo, a Ana paga três pessoas em um único lote, e o segundo pagamento é impossível:

```

BEGIN;

-- pagamento 1: Ana -> Bruno, R$ 100,00
UPDATE conta SET saldo = saldo - 100 WHERE id = 1;
UPDATE conta SET saldo = saldo + 100 WHERE id = 2;

SAVEPOINT pagou_bruno;

-- pagamento 2: Ana -> Davi, R$ 2.000,00 (a Ana não tem esse dinheiro)
UPDATE conta SET saldo = saldo + 2000 WHERE id = 4;
UPDATE conta SET saldo = saldo - 2000 WHERE id = 1;    -- ERRO: saldo -1100.00

ROLLBACK TO SAVEPOINT pagou_bruno;    -- desfaz só o pagamento 2

-- pagamento 3: Ana -> Carla, R$ 50,00
UPDATE conta SET saldo = saldo - 50 WHERE id = 1;
UPDATE conta SET saldo = saldo + 50 WHERE id = 3;

COMMIT;

```

Resultado: Ana R\$ 850,00, Bruno R\$ 600,00, Carla R\$ 250,00 e Davi R\$ 50,00 — total de R\$ 1.750,00. Dois pagamentos foram confirmados e o impossível foi recusado, sem que o lote inteiro se perdesse. Note que o ROLLBACK TO SAVEPOINT também **tira a transação do estado abortado**: depois dele, os comandos voltam a ser aceitos.

Dois detalhes costumam confundir:

- **RELEASE SAVEPOINT não confirma nada.** Ele apenas descarta o marcador; quem grava é só o COMMIT. Depois do RELEASE, tentar voltar àquele ponto resulta em ERROR: savepoint "nome" does not exist.
- **ROLLBACK sem nome desfaz a transação inteira**, com todos os pontos seguros que existirem dentro dela.

Comando	Até onde volta	A transação continua?
ROLLBACK TO SAVEPOINT s	até o SAVEPOINT s	sim
ROLLBACK	até o BEGIN	não

6.6 A ligação com a Aula 06

Na Aula 06, o bloco BEGIN ... EXCEPTION ... END do PL/pgSQL desfazia apenas os comandos do próprio bloco quando um erro era capturado. Para isso, o PostgreSQL cria um **ponto de salvamento implícito** na entrada de todo bloco que tenha tratador de exceção (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026c). É exatamente o mecanismo do SAVEPOINT, só que automático — e é por isso que a Aula 06 alertava para o custo de blocos com EXCEPTION dentro de laços longos.

6.7 Cuidados práticos

1. **Transação aberta segura linhas.** Enquanto uma transação não termina, as linhas que ela alterou ficam bloqueadas para outras alterações. Nunca deixe uma transação aberta esperando uma resposta do usuário.
2. **Confira o resultado do COMMIT.** Em uma transação abortada, ele responde ROLLBACK.
3. **Saiba em que modo está a sua ferramenta.** No Query Tool do pgAdmin, é possível desligar o *autocommit* (opção *Auto commit?*); a partir daí, cada alteração fica pendente até se clicar em **Commit** ou em **Rollback** (PGADMIN DEVELOPMENT TEAM, 2026). Se a aba for fechada com a transação aberta, o pgAdmin pergunta *Commit transaction?* e pede a escolha entre confirmar e desfazer.

7. Consistência em ambientes distribuídos e o protocolo 2PC

7.1 Quando os dados estão em mais de um servidor

A Aula 04 apresentou a **fragmentação horizontal**: as linhas de uma relação divididas entre servidores. Suponha que a Cooperativa Cerrado tenha passado a manter um servidor por agência:

Servidor	Contas
agência de Palmas	Ana, Bruno
agência de Gurupi	Carla, Davi

A transferência da Ana para o Davi agora envolve **dois SGBDs**. Cada um garante as propriedades ACID apenas **para as suas próprias transações**. Se a aplicação fizer um COMMIT em cada servidor, um depois do outro, basta o segundo falhar para que o débito exista em Palmas e o crédito não exista em Gurupi. E o primeiro COMMIT não pode ser desfeito, porque é durável.

Uma transação que altera dados em mais de um servidor é uma **transação distribuída**. Para que ela seja atômica, os servidores precisam **combinar antes** se todos vão confirmar (ÖZSU; VALDURIEZ, 2020).

7.2 Coordenador e participantes

- **Participantes** são os servidores que guardam os dados envolvidos.
- **Coordenador** é quem conduz a transação distribuída: a aplicação ou, com mais frequência, um **gerenciador de transações** (por exemplo, o de um servidor de aplicação Java, que segue o padrão XA).

A regra que o protocolo garante é: **nenhum participante confirma antes que todos possam confirmar**.

7.3 O protocolo de confirmação em duas fases (2PC)

O **protocolo de confirmação em duas fases** (*two-phase commit*, 2PC), descrito por Gray (1978), divide o término da transação em duas etapas (ELMASRI; NAVATHE, 2018; ÖZSU; VALDURIEZ, 2020).

Fase 1 — preparação (votação). O coordenador pede a cada participante que se prepare. Cada participante executa a sua parte, **sem confirmar**, e:

- se conseguiu, **grava em armazenamento durável** tudo o que é necessário para confirmar depois, mesmo que caia e volte, e vota **SIM**;
- se não conseguiu (violação de restrição, falta de espaço, erro qualquer), desfaz a sua parte e vota **NÃO**.

Votar SIM é uma **promessa**: o participante abre mão de decidir sozinho e se compromete a obedecer à decisão do coordenador.

Fase 2 — decisão. O coordenador reúne os votos e registra a decisão:

Votos recebidos	Decisão	Ordem enviada
todos SIM	confirmar	“confirme”
pelo menos um NÃO	desfazer	“desfaça”
algum participante não respondeu	desfazer	“desfaça”

Os participantes executam a ordem e respondem ao coordenador. O resultado é a atomicidade **entre servidores**: ou todos confirmam, ou todos desfazem.

7.4 O ponto fraco: o bloqueio

Entre o voto SIM e a chegada da decisão, o participante está numa **janela de incerteza**. Se o coordenador cair nesse intervalo, o participante:

- **não pode confirmar sozinho**, porque o coordenador pode ter decidido desfazer (algum outro participante pode ter votado NÃO);
- **não pode desfazer sozinho**, porque o coordenador pode ter decidido confirmar — e outro participante pode já ter confirmado.

Ele precisa **esperar**. E, enquanto espera, continua segurando os bloqueios das linhas envolvidas. Como a promessa está gravada em disco, nem reiniciar o servidor a apaga. Por isso o 2PC é classificado como um protocolo **bloqueante** (ÖZSU; VALDURIEZ, 2020). Na prática, a pendência é resolvida quando o coordenador volta e reenvia a decisão, ou por um administrador, que decide manualmente cada transação pendente.

7.5 O 2PC no PostgreSQL

O PostgreSQL implementa o papel de **participante** do 2PC com três comandos e uma visão do catálogo (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026d):

Comando	Papel no 2PC
PREPARE TRANSACTION 'id'	fase 1: grava a transação em disco e a desliga da sessão — o voto SIM
COMMIT PREPARED 'id'	fase 2: confirma a transação preparada
ROLLBACK PREPARED 'id'	fase 2: desfaz a transação preparada
pg_prepared_xacts	visão que lista as transações preparadas e ainda não decididas

Algumas regras decorrem desses comandos:

- O recurso vem **desligado**: o parâmetro `max_prepared_transactions` tem valor padrão zero e só pode ser alterado com o servidor reiniciado. O `compose.yaml` da aula inicia o PostgreSQL com `- c max_prepared_transactions=10`.
- O identificador ('id') é um texto escolhido pelo coordenador, com menos de 200 bytes, e não pode repetir o de outra transação preparada **no mesmo servidor**.
- Depois do `PREPARE TRANSACTION`, a sessão fica livre: a transação preparada não pertence mais a ela. A decisão pode ser tomada de outra sessão, mas **no mesmo banco de dados** em que a transação foi preparada; de outro banco, o servidor responde `prepared transaction belongs to another database`.
- Se a transação tiver falhado antes do `PREPARE TRANSACTION`, o comando responde `ROLLBACK`: a transação é desfeita, e esse é o voto NÃO.

Exemplo, com o laboratório simulando as duas agências como dois bancos de dados (`agencia_palmas` e `agencia_gurupi`) do mesmo servidor:

```
-- Sessão 1, banco agencia_palmas      -- Sessão 2, banco agencia_gurupi
BEGIN;                                  BEGIN;
UPDATE conta SET saldo = saldo - 300    UPDATE conta SET saldo = saldo + 300
WHERE id = 1;                           WHERE id = 4;
PREPARE TRANSACTION 'transf1-palmas';   PREPARE TRANSACTION 'transf1-gurupi';
```

Nesse ponto, as duas transações aparecem em `pg_prepared_xacts`, mas o saldo da Ana **ainda é R\$ 1.000,00**: preparada não é confirmada. Como os dois participantes votaram SIM, o coordenador envia a decisão:

```
COMMIT PREPARED 'transf1-palmas';      COMMIT PREPARED 'transf1-gurupi';
```

E o saldo da Ana passa a R\$ 700,00; o do Davi, a R\$ 350,00.

A documentação do PostgreSQL faz dois alertas importantes:

1. `PREPARE TRANSACTION` **não se destina ao uso direto por aplicações** ou por sessões interativas. Ele existe para que um **gerenciador de transações externo** coordene transações globais entre vários bancos. No laboratório, o estudante faz o papel desse gerenciador apenas para enxergar o protocolo.

2. **Não se deve deixar transações preparadas por muito tempo.** Além de manter os bloqueios, elas impedem o VACUUM de recuperar espaço. Monitorar `pg_prepared_xacts` é tarefa de administração.

7.6 Quando se abre mão do ACID: CAP e BASE

O 2PC mostra o preço da consistência forte num sistema distribuído: mensagens extras, espera e, no pior caso, bloqueio. Esse preço está no centro do **teorema CAP**, proposto por Brewer (2000): um sistema distribuído não consegue garantir ao mesmo tempo **consistência** (todos veem o mesmo dado), **disponibilidade** (toda requisição recebe resposta) e **tolerância a partições** (o sistema continua funcionando quando a rede se divide). Como partições de rede acontecem, a escolha real se dá quando elas ocorrem: ou o sistema preserva a consistência e deixa de responder, ou continua respondendo com dados possivelmente desatualizados (BREWER, 2012).

Muitos bancos NoSQL, voltados a escala horizontal muito grande, escolhem a disponibilidade e seguem o modelo **BASE**:

- **Basically Available:** o sistema responde sempre, mesmo com falhas parciais;
- **Soft state:** o estado pode mudar com o tempo, à medida que as réplicas trocam atualizações;
- **Eventually consistent:** sem novas escritas, as réplicas **acabam** ficando iguais — sem prazo garantido.

Contadores de curtidas, recomendações e carrinhos de compras toleram essa defasagem. Transferências financeiras, prontuários de saúde e registros jurídicos não: nesses domínios, onde o erro custa dinheiro, saúde ou direitos, as garantias ACID continuam indispensáveis.

8. Laboratório no iximiuz Labs

O laboratório é domiciliar e está descrito, passo a passo, no roteiro **Laboratório — Transações, ACID e 2PC no iximiuz Labs**, publicado no Google Classroom junto com este texto. Esta seção resume o ambiente e as atividades.

8.1 O ambiente

- **Turma no iximiuz Labs** (matrícula uma vez por semestre): <https://labs.iximiuz.com/trainings/aulas-bd2-2026-2-51d1c714>.
- **Playground da aula:** <https://labs.iximiuz.com/playgrounds/BD2-AULA07-ad3d665d>. A máquina já vem com as imagens do PostgreSQL 17 e do pgAdmin baixadas, os arquivos em `~/lab07` e quatro abas: **Sessão 1** e **Sessão 2** (dois terminais), **IDE** e **pgAdmin**.

O arquivo `compose.yaml` sobe dois contêineres:

Contêiner	Imagem	Detalhe importante
bd2-aula07-pg	postgres:17	iniciado com max_prepared_transactions=10, para permitir o 2PC; usuário e senha aluno; banco banco
bd2-aula07-pgadmin	elestio/pgadmin	porta 8181; e-mail aluno@bd2.unitins.br e senha aluno; servidor já cadastrado pelo servers.json

O arquivo preparar.sql cria os bancos agencia_palmas e agencia_gurupi, encerra sessões e transações preparadas esquecidas e recria a tabela conta nos três bancos. Ele roda sozinho na primeira subida do servidor e pode ser executado de novo a qualquer momento.

Todas as atividades começam pelos mesmos três comandos:

```
cd ~/lab07
podman compose up -d
podman exec -i bd2-aula07-pg psql -U aluno -d banco < preparar.sql
```

8.2 As sete atividades

A conta gratuita do iximiuz Labs oferece cerca de uma hora de ambiente por dia e destrói a máquina ao fim da sessão. Por isso o laboratório foi dividido em atividades **independentes**, de **no máximo 20 minutos**, que podem ser feitas em qualquer ordem:

Atividade	Conteúdo	Tempo
1	Tudo ou nada: BEGIN, COMMIT, ROLLBACK, a transação abortada e o <i>autocommit</i> (Seções 5.1, 6.1 a 6.4)	15 min
2	SAVEPOINT, ROLLBACK TO SAVEPOINT e RELEASE SAVEPOINT num lote de pagamentos (Seção 6.5)	15 min
3	Isolamento com duas sessões: a alteração pendente invisível e a espera pela mesma linha (Seção 5.3)	15 min
4	Durabilidade: <code>podman kill</code> no servidor e a recuperação pelo WAL (Seção 5.4)	15 min
5	2PC entre <i>agencia_palmas</i> e <i>agencia_gurupi</i> : as duas fases e o voto NÃO (Seções 7.3 e 7.5)	20 min
6	2PC com queda do servidor: a transação preparada que sobrevive e bloqueia (Seção 7.4)	15 min
7	A transação no Query Tool do pgAdmin, com o <i>autocommit</i> desligado (Seção 6.7)	15 min

8.3 Quem tem computador

O mesmo `compose.yaml` roda no Podman Desktop. Os três arquivos estão disponíveis para download:

- <https://raw.githubusercontent.com/alyssonbruno/publicacao/main/BD2/AULA07/compose.yaml>
- <https://raw.githubusercontent.com/alyssonbruno/publicacao/main/BD2/AULA07/preparar.sql>
- <https://raw.githubusercontent.com/alyssonbruno/publicacao/main/BD2/AULA07/servers.json>

No PowerShell do Windows, o redirecionamento `<` não funciona; use no lugar `podman exec bd2-aula07-pg psql -U aluno -d banco -f /docker-entrypoint-initdb.d/01-preparar.sql`, que lê o arquivo já montado dentro do contêiner.

8.4 Quem só tem o celular

Se não for possível abrir o ambiente, estude pelos resultados esperados, que estão todos no roteiro do laboratório. Um percurso de estudo sugerido:

1. Leia a Seção 6.4 e, no roteiro, a Atividade 1: anote por que o COMMIT respondeu ROLLBACK.
2. Leia a Seção 6.5 e a Atividade 2: calcule à mão os saldos finais do lote e compare com o resultado esperado.
3. Leia a Seção 7.3 e a Atividade 5: desenhe, numa folha, as mensagens trocadas entre o coordenador e as duas agências.

4. Leia a Seção 7.4 e a Atividade 6: explique, com as suas palavras, por que a linha do Bruno ficou bloqueada mesmo depois de o servidor reiniciar.
-

9. Pontos-chave

- **Transação** é uma unidade lógica indivisível de trabalho: tudo ou nada.
 - **ACID**: a atomicidade garante o tudo ou nada; a consistência, o respeito às regras; o isolamento, a não interferência entre transações; a durabilidade, a permanência do que foi confirmado.
 - No PostgreSQL, a consistência vem das restrições e dos gatilhos; o isolamento, do MVCC e dos bloqueios; a durabilidade, do **WAL**.
 - Sem BEGIN, o PostgreSQL trabalha em **autocommit**: cada comando é uma transação confirmada na hora.
 - Um erro dentro da transação a deixa **abortada**; o COMMIT, então, responde ROLLBACK.
 - SAVEPOINT e ROLLBACK TO SAVEPOINT desfazem só uma parte da transação; RELEASE SAVEPOINT apenas descarta o marcador.
 - Com dados em mais de um servidor, cada COMMIT local não basta: o **2PC** separa a preparação (voto) da decisão.
 - O voto SIM é uma **promessa durável**; se o coordenador cai, o participante fica **bloqueado** até alguém decidir.
 - No PostgreSQL: PREPARE TRANSACTION, COMMIT PREPARED, ROLLBACK PREPARED e pg_prepared_xacts, com max_prepared_transactions maior que zero.
 - O teorema **CAP** e o modelo **BASE** explicam por que alguns sistemas abrem mão da consistência imediata em troca de disponibilidade.
-

10. Referências

BREWER, Eric A. Towards robust distributed systems. *In*: ACM SYMPOSIUM ON PRINCIPLES OF DISTRIBUTED COMPUTING, 19., 2000, Portland. **Proceedings** [...]. New York: ACM, 2000. p. 7.

BREWER, Eric A. CAP twelve years later: how the “rules” have changed. **Computer**, Los Alamitos, v. 45, n. 2, p. 23-29, fev. 2012.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018. Disponível em: <https://plataforma.bvirtual.com.br>. Acesso em: 23 set. 2026.

GRAY, Jim. Notes on data base operating systems. *In*: BAYER, Rudolf; GRAHAM, Robert M.; SEEGMÜLLER, Gerhard (ed.). **Operating systems**: an advanced course. Berlin: Springer, 1978. p. 393-481. (Lecture Notes in Computer Science, v. 60).

GRAY, Jim; REUTER, Andreas. **Transaction processing**: concepts and techniques. San Mateo: Morgan Kaufmann, 1992.

HÄRDER, Theo; REUTER, Andreas. Principles of transaction-oriented database recovery. **ACM Computing Surveys**, New York, v. 15, n. 4, p. 287-317, dez. 1983.

ÖZSU, M. Tamer; VALDURIEZ, Patrick. **Principles of distributed database systems**. 4. ed. Cham: Springer, 2020.

PGADMIN DEVELOPMENT TEAM. **pgAdmin 4 documentation**: Query Tool. 2026. Disponível em: https://www.pgadmin.org/docs/pgadmin4/latest/query_tool.html. Acesso em: 23 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation**: Transactions. 2026a. Disponível em: <https://www.postgresql.org/docs/17/tutorial-transactions.html>. Acesso em: 23 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation**: SAVEPOINT. 2026b. Disponível em: <https://www.postgresql.org/docs/17/sql-savepoint.html>. Acesso em: 23 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation**: Control structures — trapping errors. 2026c. Disponível em: <https://www.postgresql.org/docs/17/plpgsql-control-structures.html>. Acesso em: 23 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation**: PREPARE TRANSACTION. 2026d. Disponível em: <https://www.postgresql.org/docs/17/sql-prepare-transaction.html>. Acesso em: 23 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation**: Write-Ahead Logging (WAL). 2026e. Disponível em: <https://www.postgresql.org/docs/17/wal-intro.html>. Acesso em: 23 set. 2026.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. **Sistema de banco de dados**. 7. ed. Rio de Janeiro: LTC, 2020. Disponível em: Minha Biblioteca.